

AEGIS-LAB — Implementation Specification

SITREP

Current State

The architecture plan is now fixed. This implementation spec converts that doctrine into a buildable blueprint covering:

- repository structure
- Python package layout
- database schema
- DAG state machine
- worker contracts
- IPC contracts
- artifact and manifest formats
- runtime profiles
- scheduler rules
- NPU sentinel interfaces
- GUI page map
- configuration files
- promotion and rollback logic

Objective

Define the exact implementation surface required to build AEGIS-LAB as a crash-tolerant, hardware-aware, heterogeneous model editing platform.

Threat Assessment

The implementation can still fail if:

- the state machine is ambiguous
 - artifacts are not content-addressed
 - promotion semantics are not atomic
 - worker responsibilities are blurred
 - configs are under-specified
 - timeout and escalation rules are not encoded as policy
 - UI state is allowed to drift from orchestrator truth
-

BATTLE PLAN

OBJECTIVE

Specify the concrete modules, tables, manifests, schemas, state transitions, and interfaces needed to implement AEGIS-LAB.

CURRENT_STATE

This specification assumes the following architectural truths are already accepted:

- UI is separate from the compute runtime
- CPU/AMX holds semantic authority
- iGPU participation is mandatory on compliant Intel heterogeneous production runs
- NPU correctness-sentinel participation is mandatory when present
- all permanent edits are copy-on-write
- all final exports are calibration-aware INT8 outputs with post-quant validation

1. REPOSITORY LAYOUT

```
/aegis-lab
├─ pyproject.toml
├─ README.md
├─ LICENSE
├─ .env.example
├─ configs/
│   ├─ app.yaml
│   ├─ logging.yaml
│   ├─ security.yaml
│   └─ runtime_profiles/
│       ├─ intel_heterogeneous_workstation.yaml
│       ├─ intel_cpu_ram_server.yaml
│       └─ gpu_heavy_reference.yaml
│   └─ scheduler/
│       ├─ stage_policies.yaml
│       ├─ device_rules.yaml
│       ├─ timeout_policies.yaml
│       └─ escalation_policies.yaml
│   └─ quantization/
│       ├─ int8_default.yaml
│       └─ calibration_policy.yaml
│   └─ verification/
│       └─ retention_thresholds.yaml
```



```

├─ numa.py
├─ thermal.py
├─ backends/
│   ├── cpu_amx.py
│   ├── avx512.py
│   ├── igpu_oneapi.py
│   ├── npu_openvino.py
│   └─ cuda_optional.py
├─ intake/
│   ├── fingerprint.py
│   ├── adapters/
│   ├── topology_manifest.py
│   └─ importers/
├─ probing/
│   ├── capture.py
│   ├── scoring.py
│   ├── suites.py
│   ├── anchors.py
│   └─ projections.py
├─ atoms/
│   ├── registry.py
│   ├── extractor.py
│   ├── cleaner.py
│   ├── validators.py
│   └─ lineage.py
├─ editing/
│   ├── planner.py
│   ├── delta_builder.py
│   ├── steering.py
│   ├── merge.py
│   ├── rollback.py
│   └─ safety.py
├─ verification/
│   ├── authority.py
│   ├── retention.py
│   ├── protected_eval.py
│   ├── kl_ppl.py
│   ├── differential.py
│   ├── moe.py
│   └─ reports.py
├─ quantization/
│   ├── calibration.py
│   ├── exporters.py
│   ├── validators.py
│   └─ int8_profiles.py
└─ sentinel/

```



```
|— architecture/  
|— schemas/  
|— operator_guides/  
|— runbooks/
```

2. CORE PACKAGE RESPONSIBILITIES

2.1 orchestrator

Owns global truth for jobs.

Responsibilities: - create job DAGs - assign stages to workers - reconcile reported state against artifact reality - enforce promotion and rollback policy - apply escalation when NPU or iGPU doctrine is unsatisfied - coordinate operator approval gates

2.2 state

Owns durable metadata and migrations.

Responsibilities: - SQLite WAL initialization - schema migrations - transactional stage state updates - row-level reconciliation APIs - resumable job enumeration

2.3 artifacts

Owns content-addressed artifact storage.

Responsibilities: - hash files and directories - create artifact records - layout run directories - manifest writing and verification - orphan cleanup - atomic promotion helpers

2.4 hardware

Owns hardware discovery and runtime placement.

Responsibilities: - discover CPU features, AMX, AVX-512, iGPU, NPU - build device topology manifest - monitor thermal, power, residency, bandwidth, queue depth - expose placement recommendations to scheduler

2.5 intake

Owns model import and architecture fingerprinting.

Responsibilities: - identify dense vs MoE vs multimodal vs experimental models - enumerate editable modules - estimate memory requirements - emit topology manifest

2.6 probing

Owns capture, scoring, and projection.

Responsibilities: - run CARE suites - collect activations - compute layer/token salience - build baseline anchors - build compressed representations for NPU sentinel path

2.7 atoms

Owns atom extraction, cleaning, and lineage.

Responsibilities: - define TargetAtom / ShieldAtom / ConfoundAtom - perform ridge cleaning and residualization - write immutable atom records - compute overlap and rejection metrics

2.8 editing

Owns edit planning and delta generation.

Responsibilities: - steering-only edits - permanent delta tensor generation - dry-run simulation - merge guardrails - rollback snapshots

2.9 verification

Owns authoritative semantic validation.

Responsibilities: - target retention scoring - protected eval - KL/PPL metrics - differential prompt comparison - MoE telemetry analysis - promotion reports

2.10 quantization

Owns calibration-aware INT8 export.

Responsibilities: - build calibration corpora - perform quantization - run post-quant validation - reject exports that erase intended edits

2.11 sentinel

Owns the NPU correctness cascade.

Responsibilities: - stage0 statistical guard - stage1 compressed semantic sentinel - timeout enforcement - escalation logic - disagreement reporting

2.12 scheduler

Owns stage placement and device doctrine enforcement.

Responsibilities: - select execution device(s) per stage - enforce CPU/AMX authority where required - enforce iGPU participation rule - enforce NPU sentinel rule when present - account for thermal, bandwidth, NUMA, and latency budgets

2.13 workers

Own worker runtime wrappers.

Responsibilities: - stage execution - heartbeat reporting - structured result envelopes - artifact handoff - retries and partial cleanup

2.14 gui

Owns operator-facing client only.

Responsibilities: - render orchestrator truth - collect operator input - show progress, telemetry, metrics, diffs, and promotion state - never host model execution directly

3. DATABASE SCHEMA

3.1 Conventions

- Use SQLite with WAL.
- All timestamps stored as ISO-8601 UTC strings.
- Use application-generated ULIDs for primary keys where practical.
- Store enum-like values as constrained text.
- Every table with mutable rows should include `created_at` and `updated_at`.

3.2 Tables

jobs

Tracks logical runs.

Fields: - `job_id` TEXT PRIMARY KEY - `project_id` TEXT NOT NULL - `job_type` TEXT NOT NULL - `status` TEXT NOT NULL - `priority` INTEGER NOT NULL DEFAULT 50 - `runtime_profile` TEXT NOT NULL - `operator_id` TEXT - `source_model_ref` TEXT NOT NULL - `topology_manifest_ref` TEXT - `active_manifest_ref` TEXT - `current_stage_id` TEXT - `requires_operator_gate` INTEGER NOT

NULL DEFAULT 0 - `failure_reason` TEXT - `created_at` TEXT NOT NULL - `updated_at` TEXT NOT NULL

stages

Tracks declared DAG stages per job.

Fields: - `stage_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `stage_name` TEXT NOT NULL - `ordinal` INTEGER NOT NULL - `status` TEXT NOT NULL - `depends_on_stage_id` TEXT - `attempt_count` INTEGER NOT NULL DEFAULT 0 - `selected_worker_type` TEXT - `selected_device_plan` TEXT - `started_at` TEXT - `completed_at` TEXT - `last_heartbeat_at` TEXT - `failure_reason` TEXT - `config_snapshot_ref` TEXT - `input_manifest_ref` TEXT - `output_manifest_ref` TEXT - `created_at` TEXT NOT NULL - `updated_at` TEXT NOT NULL

stage_attempts

Every attempt gets its own row.

Fields: - `attempt_id` TEXT PRIMARY KEY - `stage_id` TEXT NOT NULL - `attempt_number` INTEGER NOT NULL - `worker_id` TEXT NOT NULL - `status` TEXT NOT NULL - `device_plan_ref` TEXT - `log_ref` TEXT - `started_at` TEXT NOT NULL - `completed_at` TEXT - `failure_reason` TEXT - `created_at` TEXT NOT NULL - `updated_at` TEXT NOT NULL

artifacts

Content-addressed artifacts.

Fields: - `artifact_id` TEXT PRIMARY KEY - `job_id` TEXT - `artifact_type` TEXT NOT NULL - `storage_uri` TEXT NOT NULL - `content_hash` TEXT NOT NULL - `size_bytes` INTEGER - `parent_artifact_id` TEXT - `is_temporary` INTEGER NOT NULL DEFAULT 0 - `is_promoted` INTEGER NOT NULL DEFAULT 0 - `created_at` TEXT NOT NULL

manifests

High-level manifests for stages and final runs.

Fields: - `manifest_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `manifest_type` TEXT NOT NULL - `artifact_id` TEXT NOT NULL - `content_hash` TEXT NOT NULL - `schema_version` TEXT NOT NULL - `created_at` TEXT NOT NULL

device_participation

Tracks device doctrine compliance.

Fields: - `device_participation_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `stage_id` TEXT NOT NULL - `device_type` TEXT NOT NULL - `device_id` TEXT NOT NULL - `participation_mode` TEXT

NOT NULL - `required_by_policy` INTEGER NOT NULL DEFAULT 0 - `satisfied` INTEGER NOT NULL DEFAULT 0 - `details_json` TEXT - `created_at` TEXT NOT NULL

`topology_manifests`

Model topology snapshots.

Fields: - `topology_manifest_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `architecture_family` TEXT NOT NULL - `adapter_name` TEXT NOT NULL - `module_map_ref` TEXT NOT NULL - `parameter_count` INTEGER - `editable_scope_json` TEXT NOT NULL - `supports_moe` INTEGER NOT NULL DEFAULT 0 - `supports_multimodal` INTEGER NOT NULL DEFAULT 0 - `created_at` TEXT NOT NULL

`atoms`

Immutable atom registry entries.

Fields: - `atom_id` TEXT PRIMARY KEY - `atom_type` TEXT NOT NULL - `content_hash` TEXT NOT NULL - `parent_hash` TEXT - `architecture_scope` TEXT NOT NULL - `prompt_suite_hash` TEXT NOT NULL - `extraction_config_hash` TEXT NOT NULL - `validation_status` TEXT NOT NULL - `confidence_score` REAL - `artifact_id` TEXT NOT NULL - `created_by_job_id` TEXT - `created_at` TEXT NOT NULL

`atom_metrics`

Metrics for atom validation and overlap.

Fields: - `atom_metric_id` TEXT PRIMARY KEY - `atom_id` TEXT NOT NULL - `metric_name` TEXT NOT NULL - `metric_value` REAL NOT NULL - `details_json` TEXT - `created_at` TEXT NOT NULL

`calibration_sets`

Edit-aware quantization calibration definitions.

Fields: - `calibration_set_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `content_hash` TEXT NOT NULL - `artifact_id` TEXT NOT NULL - `includes_target_prompts` INTEGER NOT NULL DEFAULT 0 - `includes_matched_pairs` INTEGER NOT NULL DEFAULT 0 - `includes_protected_prompts` INTEGER NOT NULL DEFAULT 0 - `includes_hard_negatives` INTEGER NOT NULL DEFAULT 0 - `created_at` TEXT NOT NULL

`benchmark_runs`

Verification results.

Fields: - `benchmark_run_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `stage_id` TEXT NOT NULL - `benchmark_profile` TEXT NOT NULL - `authority_level` TEXT NOT NULL - `artifact_id` TEXT

NOT NULL - `summary_json` TEXT NOT NULL - `passed` INTEGER NOT NULL DEFAULT 0 - `created_at` TEXT NOT NULL

`sentinel_runs`

NPU cascade executions.

Fields: - `sentinel_run_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `stage_id` TEXT NOT NULL - `sentinel_stage` TEXT NOT NULL - `npu_present` INTEGER NOT NULL DEFAULT 0 - `model_version` TEXT NOT NULL - `feature_artifact_id` TEXT NOT NULL - `latency_ms` REAL - `verdict` TEXT NOT NULL - `timeout_level` TEXT - `details_json` TEXT - `created_at` TEXT NOT NULL

`rollback_points`

Rollback anchors.

Fields: - `rollback_point_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `label` TEXT NOT NULL - `manifest_id` TEXT NOT NULL - `artifact_id` TEXT NOT NULL - `restorable` INTEGER NOT NULL DEFAULT 1 - `created_at` TEXT NOT NULL

`operator_gates`

Human approval checkpoints.

Fields: - `operator_gate_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `stage_id` TEXT NOT NULL - `gate_type` TEXT NOT NULL - `status` TEXT NOT NULL - `decision` TEXT - `notes` TEXT - `acted_by` TEXT - `acted_at` TEXT - `created_at` TEXT NOT NULL

`hardware_snapshots`

Telemetry snapshots for analysis and scheduling.

Fields: - `hardware_snapshot_id` TEXT PRIMARY KEY - `job_id` TEXT NOT NULL - `stage_id` TEXT - `cpu_json` TEXT NOT NULL - `igpu_json` TEXT - `npu_json` TEXT - `memory_json` TEXT NOT NULL - `thermal_json` TEXT - `timestamp` TEXT NOT NULL

4. DAG STATE MACHINE

4.1 Canonical Stage Sequence

1. `intake`
2. `probe`
3. `atom_extract`
4. `atom_clean`

5. `edit_plan`
6. `edit_generate`
7. `verify_high_precision`
8. `operator_review_pre_quant`
9. `build_calibration_set`
10. `quantize_int8`
11. `verify_post_quant`
12. `operator_review_promotion`
13. `promote_export`
14. `archive_report`

4.2 Stage Status Enum

Allowed values: - `pending` - `queued` - `running` - `awaiting_operator` - `succeeded` - `failed` - `interrupted` - `blocked` - `rolled_back` - `degraded`

4.3 Transition Rules

Generic transitions

- `pending` -> `queued`
- `queued` -> `running`
- `running` -> `succeeded`
- `running` -> `failed`
- `running` -> `interrupted`
- `running` -> `awaiting_operator`
- `awaiting_operator` -> `queued`
- `awaiting_operator` -> `blocked`
- `failed` -> `queued` only if retry policy allows
- `interrupted` -> `queued` only after reconciliation
- `succeeded` -> `rolled_back` only via rollback controller

Forbidden transitions

- `failed` -> `succeeded` directly
- `queued` -> `succeeded` directly
- `pending` -> `succeeded` directly
- `blocked` -> `running` without operator or policy action

4.4 Reconciliation Rules

On restart, the reconciler must: - mark stale `running` stages as `interrupted` - validate referenced artifacts by hash - delete orphaned temporary artifacts if policy allows - reopen eligible interrupted stages as `queued` - block stages whose inputs cannot be verified

4.5 Promotion Conditions

A job may enter `promote_export` only if: - all prior required stages are `succeeded` - CPU/AMX authority checks passed - iGPU participation doctrine is satisfied - NPU doctrine is satisfied if present - post-quant validation passed - final operator gate approved

5. ARTIFACT LAYOUT

5.1 Run Directory Layout

```
/runs/{project_id}/{job_id}/
├─ manifests/
│   ├── topology_manifest.json
│   ├── stage_manifests/
│   └─ final_manifest.json
├─ configs/
│   ├── resolved_runtime_profile.yaml
│   ├── resolved_stage_policy.yaml
│   └─ resolved_thresholds.yaml
├─ logs/
│   ├── orchestrator.log
│   └─ workers/
├─ artifacts/
│   ├── source_refs/
│   ├── activations/
│   ├── projections/
│   ├── atoms/
│   ├── deltas/
│   ├── eval/
│   ├── calibration/
│   ├── quant/
│   └─ reports/
└─ exports/
    └─ final_int8_bundle/
```

5.2 Artifact Naming

Use: - `{stage_name}__{artifact_type}__{short_hash}.{ext}`

Examples: - `probe__activation_batch__a1b2c3d4.safetensors` -
- `atom_clean__target_atom__f9e8d7c6.json` -

```
edit_generate_delta_tensor__deafbeef.safetensors -  
quantize_int8_bundle__0123abcd/
```

5.3 Atomic Promotion

Promotion algorithm: 1. write final outputs into temporary export directory 2. hash all contents 3. write final manifest 4. fsync temp contents where possible 5. atomically rename temp export directory to final export path 6. mark artifacts as promoted in DB

No direct writes to final export path before hash completion.

6. MANIFEST FORMATS

6.1 Topology Manifest

Fields: - schema_version - job_id - source_model_ref - architecture_family -
adapter_name - parameter_count - editable_modules - routing_characteristics -
moe_enabled - multimodal_enabled - hardware_profile - created_at

6.2 Stage Manifest

Fields: - schema_version - job_id - stage_id - stage_name - input_artifacts -
output_artifacts - config_hash - worker_type - device_plan - device_participation -
timings - result_summary - created_at

6.3 Final Manifest

Fields: - schema_version - job_id - project_id - source_model_hash -
topology_manifest_hash - atom_hashes - delta_hashes - calibration_set_hash -
quantization_profile - high_precision_verification_summary -
post_quant_verification_summary - device_participation_summary -
operator_gate_summary - rollback_points - export_artifacts - created_at

7. IPC CONTRACTS

7.1 Transport

Primary choice: - ZeroMQ for local structured IPC

Alternative: - gRPC if typed service boundaries are preferred early

7.2 Message Types

UI -> Orchestrator

- CreateJob
- PauseJob
- ResumeJob
- AbortJob
- ApproveGate
- RejectGate
- RequestRollback
- RequestDiff
- ListJobs
- GetJobDetails

Orchestrator -> UI

- JobCreated
- JobStatusChanged
- StageStatusChanged
- GateRequested
- TelemetryUpdated
- PromotionBlocked
- PromotionSucceeded
- RollbackCompleted
- DiffReady
- ErrorRaised

Orchestrator -> Worker

- StartStage
- CancelStage
- Ping
- RequestCleanup

Worker -> Orchestrator

- StageHeartbeat
- StageCompleted
- StageFailed
- StageInterrupted
- ArtifactRegistered
- TelemetrySample
- PolicyViolation

7.3 Worker Result Envelope

Each completed or failed stage must report: - `attempt_id` - `stage_id` - `status` - `output_artifact_refs` - `device_participation_entries` - `timing_summary` - `metric_summary` - `failure_reason` - `cleanup_actions`

8. WORKER CONTRACTS

8.1 Base Worker Requirements

Every worker must support: - startup registration - health reporting - structured logging - stage-scoped working directories - interrupt-safe cleanup - idempotent retry behavior where possible

8.2 Intake Worker

Inputs: - source model reference - runtime profile

Outputs: - topology manifest - source artifact ref - editable scope map

8.3 Probe Worker

Inputs: - topology manifest - CARE suite ref - capture config

Outputs: - activation artifacts - salience metrics - projection artifacts - baseline anchors

8.4 Atom Worker

Inputs: - activation artifacts - target/shield/confound definitions - cleaning config

Outputs: - immutable atom artifacts - atom metrics - overlap reports

8.5 Edit Worker

Inputs: - topology manifest - accepted atom refs - edit config - rollback point ref

Outputs: - delta tensors - dry-run report or merge-ready edit report

8.6 Eval Worker

Inputs: - candidate artifact ref - verification profile - protected domains

Outputs: - benchmark run entries - differential report - KL/PPL metrics - MoE report if needed

8.7 Quant Worker

Inputs: - candidate artifact ref - calibration set ref - quantization profile

Outputs: - INT8 export artifacts - quant validation summary

8.8 Export Worker

Inputs: - validated post-quant artifact ref - final manifest draft

Outputs: - promoted export bundle - final manifest

8.9 Telemetry Worker

Inputs: - hardware profile - sampling policy

Outputs: - hardware snapshots - participation records - thermal alerts - bandwidth pressure alerts

9. SCHEDULER RULES

9.1 Scheduler Inputs

- runtime profile
- stage type
- model topology
- working set estimate
- hardware topology
- thermal state
- queue pressure
- operator mode (research / production)

9.2 Placement Logic

CPU/AMX mandatory stages

- atom cleaning
- permanent delta generation
- authoritative verification
- final manifest verification

iGPU participation requirement

On compliant Intel heterogeneous production runs, at least one stage entry in the job must record iGPU participation with `satisfied = 1` before promotion.

Preferred iGPU participation stages: - probe validation slice - edit consistency micro-pass - quantized validation slice - preview inference slice

NPU rule

If NPU is present and enabled by runtime profile: - sentinel run must occur during verification and post-quant validation - timeout or disagreement escalates to CPU/AMX authority - failure to execute sentinel blocks promotion unless operator explicitly downgrades the job to research-only mode

9.3 NUMA Rules

For `intel_cpu_ram_server` profile: - pin workers to NUMA nodes - co-locate memory-intensive stages with target allocation node - avoid cross-node chatter during atom cleaning and quantization - reserve isolated cores for eval where possible

9.4 Thermal Rules

For workstation profile: - reduce concurrency when package temperature exceeds threshold - prefer shorter iGPU/NPU slices under sustained heat - never sacrifice correctness authority due to throttling - if thermal degradation invalidates NPU timeout budgets, escalate to CPU/AMX

10. NPU SENTINEL SPEC

10.1 Stage 0 Statistical Guard

Inputs: - compact scalar features - layer drift summaries - routing stats for MoE if applicable

Output: - `pass` - `suspicious` - `catastrophic` - `timeout`

Target latency: - 1–3 ms typical - 5 ms hard cap

10.2 Stage 1 Compressed Semantic Sentinel

Inputs: - CPU/AMX-generated projected vectors - edit-mode metadata - baseline anchor deltas

Output: - `pass` - `ambiguous` - `fail` - `timeout`

Latency policy: - 4–8 ms target - 15–20 ms hard cap - 35 ms fail-close absolute cap

10.3 Escalation Policy

Escalate to CPU/AMX if: - stage0 verdict is `catastrophic` - stage1 verdict is `ambiguous` or `fail` - any NPU stage times out - disagreement occurs across repeated runs

10.4 Sentinel Model Versioning

Store: - sentinel model version - projection config hash - threshold policy hash - training dataset hash - edit mode scope

These must be captured in `sentinel_runs.details.json` and referenced in the final manifest.

11. GUI PAGE MAP

11.1 Main Pages

- Dashboard
- Models
- Jobs
- Job Detail
- Probe Explorer
- Atom Registry
- Edit Planner
- Verification
- Quantization
- Exports
- Settings

11.2 Job Detail Subtabs

- Overview
- Stages
- Telemetry
- Device Participation
- Artifacts
- Diffs
- Logs
- Operator Gates
- Rollback

11.3 Required Widgets

- stage progress timeline
- DAG graph view
- live CPU/iGPU/NPU charts
- artifact browser
- before/after response diff pane
- metric summary cards
- policy compliance banner

- operator approval modal

11.4 Source of Truth Rule

Every displayed status must come from orchestrator APIs. The GUI may cache views but must not invent or infer authoritative state.

12. CONFIGURATION SCHEMAS

12.1 app.yaml

Fields: - environment - storage_root - db_path - ipc_transport - log_level - default_runtime_profile - ui_refresh_ms

12.2 runtime_profiles/*.yaml

Fields: - profile_name - production_mode - cpu_authority_required - igpu_required - npu_required_if_present - numa_enabled - thermal_limits - preferred_backends - max_concurrent_workers - spill_to_disk_enabled

12.3 device_rules.yaml

Fields: - igpu_participation_required - igpu_valid_modes - npu_sentinel_required_if_present - cpu_authority_stage_names - promotion_block_on_missing_device_participation

12.4 timeout_policies.yaml

Fields: - worker_heartbeat_timeout_ms - stage0_npu_soft_ms - stage0_npu_hard_ms - stage1_npu_soft_ms - stage1_npu_hard_ms - stage1_npu_failclose_ms - operator_gate_timeout_ms

12.5 retention_thresholds.yaml

Fields: - target_retention_min - protected_drift_max - kl_max - ppl_delta_max - post_quant_retention_min - max_npu_disagreement_rate

12.6 calibration_policy.yaml

Fields: - require_target_prompts - require_matched_pairs - require_protected_prompts - require_hard_negatives - min_samples_per_bucket - quant_backend

13. PROMOTION AND ROLLBACK LOGIC

13.1 Promotion Controller

Algorithm: 1. verify final stage prerequisites 2. verify final artifact hashes 3. verify CPU/AMX authority pass 4. verify iGPU participation rule 5. verify NPU rule if present 6. verify operator gate approval 7. write final manifest 8. atomically promote export 9. mark rollback point

13.2 Rollback Controller

Algorithm: 1. resolve requested rollback point 2. verify rollback point manifest hash 3. verify artifact presence 4. restore active manifest pointer 5. mark superseded artifacts as non-active 6. write rollback event log

Rollback must not rely on inverse tensor arithmetic alone. It must restore a known-good manifest-defined state.

14. TEST STRATEGY

14.1 Unit Tests

Cover: - hashing - manifest serialization - state transitions - scheduler rule evaluation - timeout parsing - atom immutability logic

14.2 Integration Tests

Cover: - orchestrator + worker lifecycle - stage execution and artifact registration - operator gate handling - quantization path - promotion flow

14.3 Recovery Tests

Cover: - OOM-like worker death mid-stage - stale running rows on restart - corrupted temporary delta artifact - interrupted post-quant verification - NPU timeout escalation

14.4 Promotion Tests

Cover: - missing iGPU participation blocks promotion - missing NPU participation blocks promotion when present - failed post-quant validation blocks promotion - operator rejection blocks promotion

14.5 Performance Tests

Cover: - stage0 and stage1 NPU latency budgets - NUMA placement gains on CPU server - WAL recovery timing - large artifact hashing throughput

15. IMPLEMENTATION ORDER

Milestone 1 — Orchestrator and State Core

Build: - SQLite WAL state layer - job/stage schema - orchestrator service - worker IPC skeleton - basic CLI and API

Milestone 2 — Artifact and Manifest Core

Build: - artifact store - hashing - manifest schema and writers - atomic promotion helpers

Milestone 3 — Intake and Scheduler Core

Build: - model fingerprinting - hardware discovery - runtime profile loader - device doctrine evaluator

Milestone 4 — Probe and Atom Stack

Build: - CARE suite runner - activation capture - atom extraction and cleaning - immutable atom registry

Milestone 5 — Edit and Verification Stack

Build: - steering path - delta builder - authoritative verification - differential reporting - MoE telemetry

Milestone 6 — NPU Sentinel and iGPU Doctrine

Build: - stage0 guard - stage1 semantic sentinel - timeout enforcement - participation recording

Milestone 7 — Quantization and Promotion

Build: - calibration corpus builder - INT8 exporter - post-quant validation - final promotion and rollback

Milestone 8 — GUI Completion

Build: - dashboard - job detail pages - operator gates - diffs, telemetry, and export views

16. HARD NON-NEGOTIABLES

1. UI never owns runtime execution.
 2. Source models are read-only.
 3. Deltas are copy-on-write only.
 4. WAL is mandatory.
 5. Stage completion requires artifact verification.
 6. Atom registry is immutable and hash-linked.
 7. CPU/AMX owns semantic authority.
 8. iGPU doctrine must be satisfied for compliant production runs.
 9. NPU sentinel doctrine must be satisfied when present.
 10. NPU timeout fails closed.
 11. INT8 export is calibration-aware.
 12. Final promotion requires post-quant verification.
 13. Rollback restores manifest-defined state.
 14. GUI cannot invent state.
 15. Recovery logic is part of the core build, not a later enhancement.
-

17. FINAL POSITION

This implementation spec turns AEGIS-LAB from an abstract architecture into a concrete system blueprint.

It defines: - what packages exist - what each package owns - what tables exist - what the state machine permits - what manifests must contain - what workers return - what promotion requires - how NPU and iGPU doctrine are encoded - and how the system recovers when something inevitably dies under load

That is the actual build surface.